INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY
HYDERABAD

Assignment 05: Project SkyVault

Giving Your Officer a Memory and a Standard

Course: Agentic AI: From Concepts to Practice

Instructor: Prof. Karthik Vaidhyanathan

TAs: Aviral Gupta, Aneetta Sara Shany, Ch Pavan Harshit, Shreyash Chandak

Topic: Agent Memory & the Model Context Protocol

Deadline: 6th September 2026, 11:59 PM

Mission Briefing

TO: Lead AI Engineer
FROM: Director of Flight Operations, AeroWing Airlines
SUBJECT: FlightOps Delivers. It Also Forgets, and Shares With No One.

FlightOps has been live for a week. The tool-calling core performs as designed – multi-step questions get grounded answers, and the plan-and-reflection output has earned genuine trust from Operations. Two gaps have surfaced just as quickly.

First: FlightOps retains nothing across a restart. A gate agent's stated preference, a pilot's maintenance note – gone the moment the process exits, and re-explained from scratch every session. It reasons well *within* a conversation and remembers nothing *between* them.

Second: the Maintenance and Gate Management teams have each separately asked for access to "the FlightOps tools." Today those six functions live only inside your `agent.py`, reachable by nothing else. The only way to grant access is to hand over a copy of your code and hope it never drifts from the original.

Your Mandate: Upgrade FlightOps into **Project SkyVault** – give it memory that survives a restart, and put its tools behind a standard protocol interface that any team's agent, not just yours, can call.

Learning Objectives

By the end of this assignment you should be able to

- Explain the difference between memory that lives inside one conversation's context window and memory that persists across process restarts, and implement the latter by hand.
- Detect and resolve a conflicting memory (a fact restated with a new value), and justify the conflict-resolution rule you chose.

- Explain the interoperability problem MCP solves, and why wiring tools directly into one agent's codebase does not scale to a second consumer.
- Implement the core of the Model Context Protocol's message lifecycle (`initialize`, `tools/list`, `tools/call`) by hand, without an MCP SDK.
- Refactor an existing agent so every tool call is mediated by a standard protocol boundary instead of a direct function import.
- Articulate, from firsthand experience, what a protocol like MCP standardizes and what it deliberately leaves unstandardized (identity, state, negotiation – the gap A2A exists to fill).

Assignment Tasks

A Note Before You Start: You Are Extending, Not Rewriting

This assignment builds directly on your Assignment 4 submission. Bring forward `data.py`, `tools.py`, `schemas.py`, `agent.py`, `planner.py`, `reflector.py`, and `config.py`. You will be **extending and refactoring** this codebase, not starting a new one. If your Assignment 4 implementation was incomplete, bring it to a working baseline first – the agent loop, planner, and reflector from that assignment are assumed to work before you touch anything in this one.

This is still **not** a Google Colab notebook. Submit a ZIP of a runnable Python project, exactly as in Assignment 4.

Model/provider choice: As before, you may use any language model API that supports tool/-function calling. You are not required to change providers from Assignment 4.

IMPORTANT NOTE: We have observed a large number of fully AI-generated submissions. While the usage of AI is not prohibited, the assignments are meant to be done by the learner primarily. Please use LLMs/AI Agents only as learning aids. As a proof of your effort, please mention your roll number as a comment on top of every python file you make in the project, and also in the README.

Part 1: Giving SkyVault a Memory

Objective: Make SkyVault remember facts across process restarts, not just within one run.

Everything SkyVault "remembers" today lives in a single conversation's context – close the program and it is gone. In `memory.py`, build a small persistent memory store:

1. Implement `remember(key, value, source)` and `recall(query)` functions backed by a file on disk (a JSON file is sufficient – you do not need a database). Memory written in one run of the program must still be there the *next time the program starts.*
2. At the start of every run, load a short summary of everything currently in memory and fold it into the system prompt, the way MemGPT's *working context* sits alongside its conversation queue. SkyVault should not have to be explicitly asked to recall a fact it already knows.
3. Also register `remember` and `recall` as tools the model can call mid-conversation, for facts learned or needed *during* the current exchange (not just at startup).
4. Handle conflicting facts. If a stored fact is restated with a different value (see the example below), your implementation must not silently keep both versions side by side – decide on a conflict rule (e.g. same key overwrites the old value) and state it explicitly in your README.

You do not need Mem0's full extract/update pipeline; you do need a rule, and you need to have actually implemented it rather than left it to chance.

Worked example

```
Session 1:
  User: "I usually work Terminal 2, remember that."
  SkyVault: "Got it -- I'll keep that in mind."
  [process exits]

Session 2 (fresh restart):
  User: "Find me an open gate."
  SkyVault: (recalls "prefers Terminal 2" from memory)
            calls find_available_gate("Terminal 2")
            "Gate B12 is open in Terminal 2."
```

Demonstrate this end to end: run SkyVault, state a fact, exit the process completely, restart it, and ask a question that can only be answered correctly if that fact was actually recalled – not re-typed by you.

Critical Incident Report

Memory solved one half of the Director's memo. The other half is still open: two different teams have now separately asked for access to your tools, and both conversations ended the same way – an engineer offering to email over a copy of `tools.py`.
This is precisely the interoperability problem covered in the fortnight's material. Three teams, six tools, each wired bespoke, is already an $M \times N$ mess waiting to happen – and every copy that drifts from your original is a second maintenance burden and a second place for a bug to hide. The fix is not to write faster copy-paste; it is to put a standard interface between your tools and whoever wants to call them.

Part 2: Speaking MCP

Objective: Implement the core MCP message lifecycle by hand, so a tool call is mediated by a standard protocol instead of a direct function import.

You are not going to install an MCP SDK for this. You are going to build the three message types that make up MCP's tool lifecycle yourself, exactly as covered in the lecture material – plain Python dictionaries shaped like JSON-RPC, passed directly between your own objects. You do not need real network transport (sockets, HTTP, stdio) for this assignment; two Python objects calling each other's methods with correctly-shaped dictionaries is sufficient.

Every request must carry a `jsonrpc` version, a `method`, its `params`, and an `id`. Every response must carry back the *same* `id`, and either a `result` or an `error`.

```
# Request
{"jsonrpc": "2.0", "id": 3, "method": "tools/call",
 "params": {"name": "find_available_gate",
            "arguments": {"terminal": "Terminal 2"}}}

# Response
{"jsonrpc": "2.0", "id": 3,
 "result": {"gate": "B12", "terminal": "Terminal 2"}}

# Error response (same id, no result)
{"jsonrpc": "2.0", "id": 3,
 "error": {"code": -32001, "message": "Unknown tool: find_gate"}}
```

Implement all three methods:

1. `initialize` – a handshake exchanging client/server name and protocol version.
2. `tools/list` – returns the name, description, and parameter schema for every registered tool.
3. `tools/call` – executes a named tool with the given arguments and returns its result, or a JSON-RPC error object if the tool name is unknown or a required argument is missing.

Part 3: Standing Up the MCP Server

Objective: Wrap your existing tools behind a standard protocol boundary.

In `mcp_server.py`, build:

1. A **Tool Registry** that wraps every function you already wrote in `tools.py` – including `remember` and `recall` from Part 1 – with its name, description, parameter schema, and required fields, exactly as you did in `schemas.py` for Assignment 4.
2. An **MCPServer** class that handles incoming `initialize`, `tools/list`, and `tools/call` messages by delegating to the registry, and returns correctly-shaped JSON-RPC responses – including error objects for bad requests.

Keep the registry and the server as separate classes. The registry should not know or care how its tools are exposed; the server should not know or care how a tool is implemented internally.

Part 4: The MCP Client, and Retiring the Direct Import

Objective: Make your agent talk to its tools only through the protocol.

In `mcp_client.py`, build an **MCPClient** with a `list_tools()` method and a `call_tool(name, arguments)` method, each of which sends a correctly-shaped request to your `MCPServer` and returns the parsed result.

Then refactor `agent.py`:

- `agent.py` must no longer `import tools` directly. Every tool invocation goes through your `MCPClient`.
- The tool declarations you hand to the model at the start of a run must now come from calling `list_tools()` on the client – not from a hardcoded list copied into `agent.py`.

Your agent effectively becomes the **Host** (lecture material reference if you've been paying attention)

it owns the LLM and decides when a tool is needed, but it never touches `tools.py` directly – the **Client** inside it does that, by talking to your **Server**.

Exploration (Not Graded)
Write a second, tiny script – `gatebot.py`, ten to fifteen lines – that imports *only* `mcp_client.py` (not `agent.py`, not `tools.py`) and calls `find_available_gate` through it directly. If this works without touching a single line of your agent's code, you have just proven, concretely, the payoff the Director asked for: a tool built once, reused by a consumer you didn't write.

Part 5: Proving It Still Works

Objective: Verify memory and MCP together, end to end.

1. Re-run a multi-tool question from Assignment 4 (or a similar one) and confirm the full trace – Goal → Plan → `tools/list` → `tools/call` (several times) → Answer → Reflection – still works with every tool call now routed through MCP.
2. Confirm memory persists across a full process restart *and* that a recalled fact can be used as an argument to an MCP `tools/call` in the same run (as in the Part 1 worked example).
3. Log every JSON-RPC request/response pair for at least one full multi-tool run, and include the log in your submission (either printed clearly to the console and captured in a screenshot, or saved to a trace file referenced from your README).

Constraints: Build It Yourself, Again

No **LangChain**. No **CrewAI**. No **AutoGen**. No agent frameworks of any kind – same as Assignment 4.
This time, also: **no MCP SDK** (`pip install mcp` or equivalent). The message shapes in Part 2 must be your own dictionaries, handled by your own code. You are meant to understand exactly what the protocol is standardizing – and exactly what a real SDK would have hidden from you – before you ever let one do it for you.

Final Report

Answer the following in your `README.md`. Each answer should be three to five sentences.

1. Your agent had two different kinds of "forgetting" before this assignment: forgetting mid-conversation (context window limits) and forgetting between sessions (no persistence at all). What did `memory.py` actually fix, and what would still break if AeroWing rolled this out to ten thousand pilots at once?
2. Before this assignment, a call to `find_available_gate` was a plain Python function call your agent trusted implicitly. After Part 3 and 4, that call has to cross a protocol boundary. What changed about how much the server has to trust the caller, versus how much the caller has to trust the server – and where would you add a check if a malicious or buggy client started sending it requests?

Final Report

3. Suppose AeroWing next asks SkyVault to hand off a task to a `MaintenanceAgent` built by a different team, on a different LLM, that needs to ask clarifying questions and negotiate before it responds – not just return a value. Would the MCP layer you just built be sufficient for that hand-off? Name the protocol from this fortnight's material that actually fits, and identify the one capability MCP lacks that makes it necessary.

4. You have now built the tool loop, planner, reflector, memory, and an MCP layer entirely by hand, across two assignments. Pick one of these that a framework like CrewAI or AutoGen would have given you for free, and explain concretely what you would have lost in understanding by starting with the framework instead of building it yourself first.

Submission Guidelines

- Submit a single zip file consisting of
  - your complete Python project: `data.py`, `tools.py`, `schemas.py`, `agent.py`, `planner.py`, `reflector.py`, `memory.py`, `mcp_server.py`, `mcp_client.py`, `main.py`, `config.py`, plus `gatebot.py` if you attempted the exploration
  - your `README.md`, including a short note on what changed since Assignment 4, your memory conflict-resolution rule, run instructions, example prompts, and your Final Report answers
- Ensure your project runs from the command line, end to end, without errors on a clean check-out.
- Configure your chosen model provider through environment variables, loaded via `config.py`. Do **not** hardcode it, and do **not** include your `.env` file in the submission – include a `.env.example` instead.
- Submit the file using the filename:

`SkyVault_YourName.zip`

A colleague who forgets everything by morning is exhausting to work with.

Give SkyVault a memory, and a standard door for others to knock on, and it stops being your project – it becomes AeroWing's infrastructure.